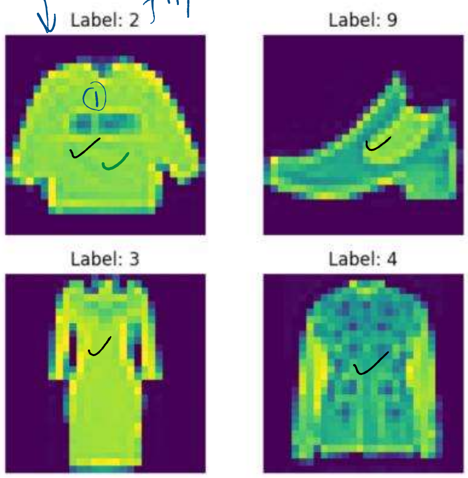
Recap

02 January 2025 16:49

flip rotate ML

fashion MNIST



→ pytorch

↓

ANN

60000 images

stage 1

cpu → train

5000 images → 82-83%

stage 2

gpu - train

→ 60000 → $\boxed{88}$

The Problem

02 January 2025 16:50

test → 88%
train → 98%
→ 10%
↓
overfitted
↓
reduce

test → 88%
train → 93%
→ $\boxed{5\%}$

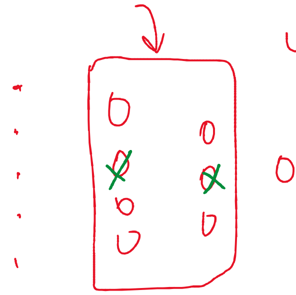
Solution

02 January 2025 16:50

$\boxed{60000}$

Loss + penalty

1. Adding more data ✓ ✗
2. Reducing the complexity of NN architecture ✗
3. Regularization ✓
4. Dropouts ✓
5. Data Augmentation → CNN ✗
6. Batch Normalization ✓
7. Early Stopping ✗



optimize

→ Regularization

→ Dropouts

→ Batch Norm

→ train → $\frac{\text{train}}{\text{test}}$ → gap reduce

Dropout

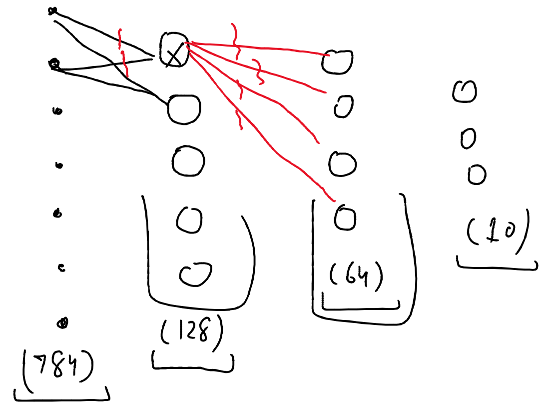
03 January 2025 18:22

$[p = 0.5]$ 50%

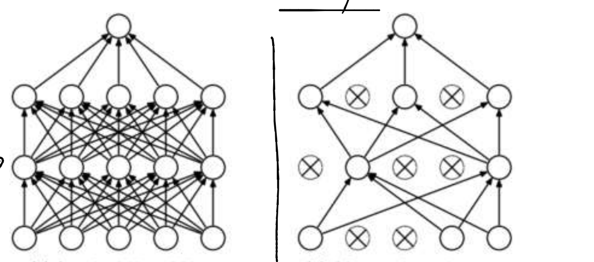
0.3

1. Applied to the hidden layers
2. Applied after the ReLU activation function
3. Randomly turns off p% neurons in the hidden layer during each forward pass
4. This has a regularization effect ✓
5. During evaluation dropout is not used

each forward pass



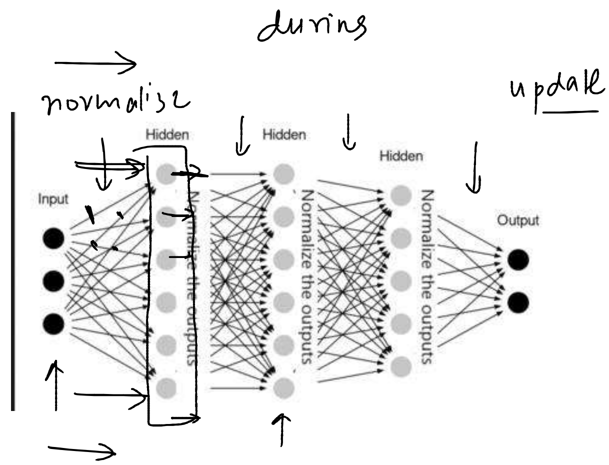
simplify



(a) Standard Neural Net (b) After applying dropout.

Batch Norm

03 January 2025 18:40



- Applied to Hidden Layers:
    - Typically applied to the hidden layers of a neural network, but not to the output layer.
- Applied After Linear Layers and Before Activation Functions:
    - Normalizes the output of the preceding layer (e.g., after nn.Linear) and is usually followed by an activation function (e.g., ReLU).
- Normalizes Activations:
    - Computes the mean and variance of the activations within a mini-batch and uses these statistics to normalize the activations.
- Includes Learnable Parameters:
    - Introduces two learnable parameters, gamma (scaling) and beta (shifting), which allow the network to adjust the normalized outputs.
- Improves Training Stability:
    - Reduces internal covariate shift, stabilizing the training process and allowing the use of higher learning rates.
- Regularization Effect:
    - Introduces some regularization because the statistics are computed over a mini-batch, adding noise to the training process.
- Consistent During Evaluation:
    - During evaluation, BatchNorm uses the running mean and variance accumulated during training, rather than recomputing them from the mini-batch.

L2 Regularization

03 January 2025 18:58

optimization

$L \to$ mse

$L \to$ logloss $\to$ weights/biases

Applied to Model Weights:

- Regularization is applied to the weights of the model to penalize large values and encourage smaller, more generalizable weights.

Introduced via Loss Function or Optimizer:

- Adds a penalty term λ∑wi2 to the loss function in L2 regularization.

$$\text{Loss}_{\text{reg}} = \text{Loss}_{\text{original}} + \lambda \sum w_i^2$$

$$\lambda \left(w_1^2 + w_2^2 + w_3^2 + w_4^2 \right)$$

- In weight decay, directly modifies the gradient update rule to include λwi, effectively shrinking weights during training.

$$\boxed{w \leftarrow w - \eta(\nabla \text{Loss} + \lambda w)}$$

weight_decay

Penalizes Large Weights:

- Encourages the network to distribute learning across multiple parameters, avoiding reliance on a few large weights.

Reduces Overfitting:

- Helps the model generalize better to unseen data by discouraging overly complex representations.

Controlled by a Hyperparameter:

- A regularization coefficient (λ often set via weight_decay in optimizers) controls the strength of the penalty. Larger values lead to stronger regularization.

No Effect on Bias Terms:

- Regularization is typically applied only to weights, not biases, as biases don't directly affect model complexity.

Active During Training:

- Regularization affects weight updates only during training. It does not explicitly influence the model during inference.